

Autor: Ing. Klever Guamushig

Url Github: https://github.com/MauriGJaque/CitaSalud_service

Url GitHub: <https://github.com/MauriGJaque/CitaSalud-controlDocente>

REFLEXIÓN DE APRENDIZAJE (CITASALUD-SERVICE)

1. ¿Qué cambió en mi forma de "dar por terminado" el código cuando el veredicto lo decidió un gate determinista en vez de mi propio criterio?

Cambió en que ya no puedo confiarme solo de lo visual. Antes, si yo programaba una pantalla, veía que el botón guardaba la cita médica y no salía ningún error en la consola, para mí el trabajo ya estaba terminado.

Con el "gate" automático todo cambió, porque el sistema no trabaja con corazonadas. Si la guía de diseño (spec.md) pide una regla, como el control de que dos personas no reserven la misma cita a la vez, el gate me exige por obligación escribir una prueba automática para ese caso. Si no ve la prueba corriendo, simplemente me bloquea el paso. Ahora, dar por terminado el código significa que debo demostrar con datos reales y pruebas que todo funciona exactamente como se pidió.

2. ¿Qué pilar me costó más dejar en verde —pruebas, seguridad o criterios—, y por qué?

El de **Seguridad con Semgrep**. Al principio me costó bastante entenderlo solo, pero gracias a que usted nos fue guiando paso a paso en la clase, le agarré el hilo y pude resolverlo. Lo más estresante fue el tema del **tokens** de Claude Code; como se consumen rápido con cada escaneo de seguridad y corrección en los archivos de Java, me daba miedo quedarme sin saldo a mitad del ejercicio. Al final, siguiendo su explicación, logré configurar bien el entorno y pasar el pilar cuidando los recursos que me quedaban.

3. ¿Para qué me serviría un gate de Definition of Done (y el escaneo automático de seguridad vía MCP) en mi equipo real?

En mi trabajo diario con sistemas web e institucionales, esto nos serviría bastante para evitar errores humanos por el apuro de las entregas.

A veces, por el tiempo, se sube código sin probarlo por completo o se dejan configuraciones inseguras sin darse cuenta. Si tuviéramos este inspector automático en nuestro equipo, ningún programador podría subir un cambio que dañe lo que ya funciona. Además, el escaneo de seguridad nos avisará de cualquier error crítico en el momento exacto, ahorrándonos horas de revisiones manuales y dándonos la tranquilidad de que el software que sale a producción es estable y seguro.